openAG Operational Pipelines (B-ROUND + B1–B7)

one fixpoint engine, seven entry-point behaviors, ~33 behaviors

openAG (architecture-model-standard — gen-0 seed)

2026-10-04

Overview

openAG has **one engine** — the `B-ROUND` least-fixpoint — and **seven behaviors** (`B1`–`B7`) that are *entry points* into it. Unlike AMS's seven flat sibling pipelines (`pipelines-v3.md`), these are **not** siblings: six of them feed or read a single shared loop; one (`B7`) *is* a round stage run standalone.

Source: `docs/plans/2026-10-04-openAG-pipelines.md` (design), `2026-10-04-openAG-entity-model.md` (gen-0 ledger), `2026-10-04-openAG-attribute-corpus.md` (provenance). Companion to AMS `docs/reports/pipelines-v3.md` — this is the openAG-framed analog.

Behavior	Name	Role	Steps	Realizes
B-ROUND	`discharge_open_promises_to_engine`	**the** `B-ROUND` **fixpoint**	6	all 5 caps
B1	`extract_seed_graph`	drives rounds	8	CAP-extract
B2	`author_promise_content`	drives rounds	5	CAP-model
B3	`re_extract_incremental`	drives rounds	4	CAP-extract
B4	`represent_artifacts`	**reads** graph	5	CAP-represent
B5	`generate_code_mdd`	**reads** graph	4	CAP-mdd
B6	`learn_and_tune_rules`	drives rounds	4	CAP-learn
B7	`gate_standalone`	**is** `B-ROUND.gate`	2	CAP-model

Coverage (gen-0, from corpus §11): ~33 behaviors · ~86 verified / ~34 delivered / ~9 typed-gap primary promises. openAG **drops exactly one** AMS pipeline (legacy 10-stage batch) and absorbs the other four + three cross-cutting services into the fixpoint (zero-loss crosswalk, §4).

Legend

Each stage carries a **seam marker** (the neurosymbolic split, design §3.4) and a **status** overlay. **Solid** = grounded in forked AMS code today. **Starred** (∗) = net-new openAG design intent (the invention surface, corpus §13).

Marker	Meaning
`SYM`	Symbolic stage — deterministic; *may* write the graph
`NEU`	Neural seam — may invent/score candidates; **never** writes directly
`drives`	Feeds residual open promises into `B-ROUND`
`reads`	Reads the converged graph; never grows it
`A1..A7`	The axis a promise/stage operates on (abstraction…evidence)
∗ suffix	Net-new (no prior AMS source) — typed-gap in the corpus
·	No marker at this step

Axis glossary (the 7 discharge axes)

Axis	Name	Discharges
A1	Abstraction	roll-up / decomposition (↑ ↓)
A2	Realization	capability → component realizes
A3	Boundary	interfaces, dependency direction
A4	Governance	constraints, layers, budget, scheduling
A5	Flow	produces / consumes / triggers
A6	Lifecycle	generations, identity, caching
A7	Evidence	grounding against typed evidence

Capability glossary (CAP)

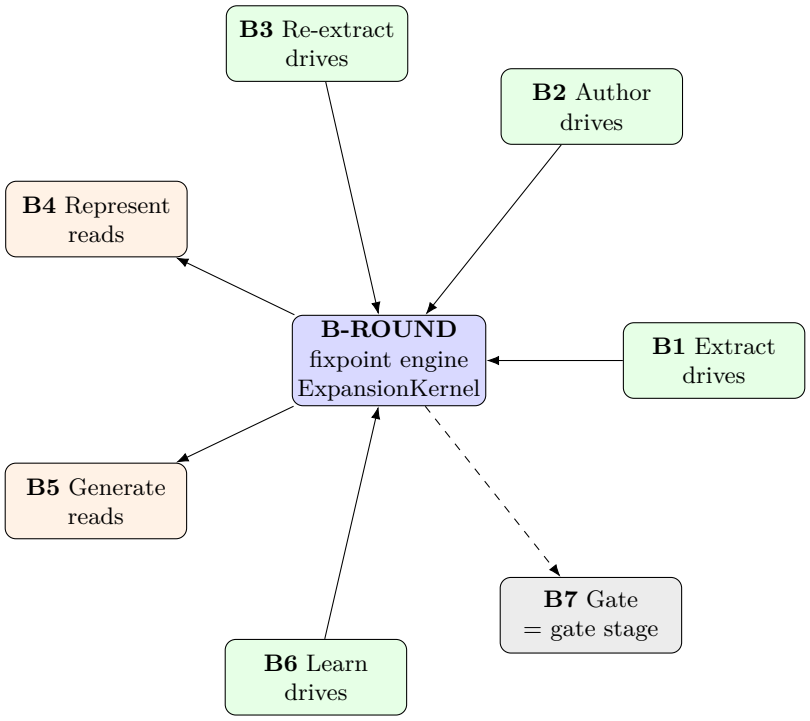
ID	Description	AMS crosswalk
CAP-extract	Seed nodes + grounded facts via pure AST, no LLM	CAP-PB + pipeline_v3
CAP-model	Grow graph to least-fixpoint by discharging promises	CAP-DF + CAP-WK
CAP-represent	Materialize / project / render owned views	CAP-SL + lifecycle
CAP-learn	Observe outcomes, distill lessons, tune rules	BEH-27 + BEH-FBK
CAP-mdd	Model-driven codegen + behavioral parity	blind-regen campaign

Component glossary (COMP)

ID	Description
ExpansionKernel	Coordinator of the fixpoint; runs B-ROUND
ExpanderRegistry	Total (kind,axis) → expander registry (∗ net-new)
PromiseEngine	Computes open promises per node from kind-valence
Grounder	Checks candidates against typed evidence (∗ net-new)
GateEvaluator	Admits schema+constraint-valid state; emits gaps
Reconciler	Closure operator: dedup, identity-bind, roll-up
BudgetGovernor	Critical-path, tier-exhausting, cost-aware scheduler
IdentityStore	Sticky θ-crosswalk; stable IDs across generations (∗)
GenerationStore	Publishes + versions model generations
WalkEngine	Runs declarative WalkSpecs (the operator prototype)
SourceScanner	AST-walk target tree → observable-fact inventory
PatternDetector	Endpoint/cluster detection
Seeder	Emits seed nodes with kind-valence open promises
Slicer	Materializes ModelSlice contracts into closed fragments
ProjectorRegistry	Named pure projectors → DiagramSpec
Renderers	Deterministic md/html/svg/zip/ai-context renderers
Viewer	Self-contained HTML promise-graph lenses
FeedbackJournals	Append-only JSONL streams (gates/drift/test/llm)
LessonStore	Deduplicated lesson store (content-hashed IDs)
RuleTuner	Tunes expander/scheduler rules from lessons
CodeGenerator	Generates code from model; oracle-in-loop
ParityChecker	Runs oracle, diffs, scores behavioral parity
LLMProviderRouter	Provider-agnostic routing (policy, budget, fallback)

The engine at a glance

`B-ROUND` is the hub; `B1`–`B7` are the entry points. Six behaviors feed or read it; `B7` *is* its `gate` stage invoked alone.

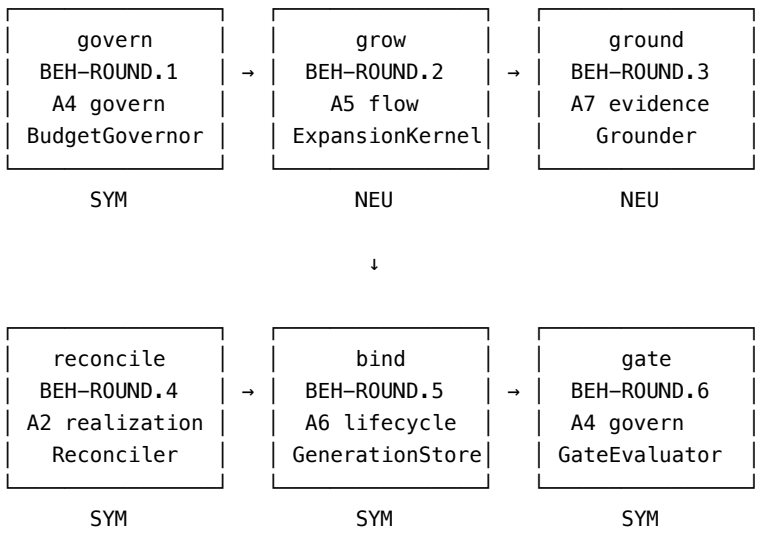


B-ROUND — discharge_open_promises_to_fixpoint

One round is `govern → grow → ground → reconcile → bind → gate`. Rounds repeat until no open promise remains within budget. Because `reconcile` is a closure operator, the run is **confluent modulo budget**: scheduling order changes *which gaps appear under a budget cut*, never the canonical grounded form.

Realizes: all 5 capabilities (the shared operator)

Flow



* loopback: gate → govern while open promises remain within budget

Stage I/O

#	stage	consumes (←)	produces (⇒ owned I/O)	seam
1	**govern**	open-promise worklist, `CON-parametric` budgets	`JOB-*` per scheduled discharge	SYM
2	**grow**	selected `Promise`, ExpanderRegistry `(kind,axis)`	candidate set (in-flight)	NEU
3	**ground**	candidate set, `EvidenceRef`, Target A7 signal	`Evidence` nodes (deduped)	NEU
4	**reconcile**	grounded candidates, IdentityStore	mutates `ModelGraph`	SYM
5	**bind**	reconciled delta	`GEN-*`, `DEC-*`	SYM
6	**gate**	committed graph, `CON-existence/prohibition/parametric`	`JRN-gates` + `GateEvent`; `typed-gap`	SYM

Promises

- **intent** *(delivered)* — Grow the graph to least-fixpoint by discharging open promises along their axes. `[plan]` design §operator `[model]` CAP-DF intent.
- **success_criteria** *(delivered)* — Fixpoint reached modulo budget; confluent reconcile; every discharge grounded or emitted as typed-gap. `[plan]` design invariant #5.

- **requirements** *(verified)* — Substrate exposes Stage/Pipeline/ ContentCache/silent-skip; kind+predicate selection; attribute-category- aware rollup (A=union, B=conjunction, C=contract). `[model]` CAP-WK requirements; `walks/`, `pipeline_v3/substrate/`.
- **termination (invariant #2)** *(delivered)* — Every discharge grounds in finite evidence or emits a gap; budget widening empties the worklist. `[plan]` design §3.3.
- **gap-honesty (invariant #5)** *(delivered)* — Anything ungrounded within budget becomes a typed gap, never silently omitted. `[plan]` design `[discussion]` L13.

Round stages (6)

ID	Name	Intent
`BEH-ROUND.1`	`govern`	Build discharge schedule: critical-path DAG over open promises, slack, tier-exhaust (T1 symbolic first), widen when blocked.
`BEH-ROUND.2`	`grow`	Select next open promise; expander (symbolic/neural) proposes candidate edges/nodes along the promise's axis.
`BEH-ROUND.3`	`ground`	Check candidates against typed evidence (static/observational/executable); reject or downgrade ungrounded to a gap; score confidence.
`BEH-ROUND.4`	`reconcile`	Closure operator: dedup, identity-bind via θ-crosswalk, roll-up aggregation, canonicalize — the join of the lattice.
`BEH-ROUND.5`	`bind`	Commit stable IDs (sticky across generations); record the new revision; no silent re-binding.
`BEH-ROUND.6`	`gate`	Admit only schema+constraint-valid state; evaluate the 3 verification classes; emit residual as typed gap.

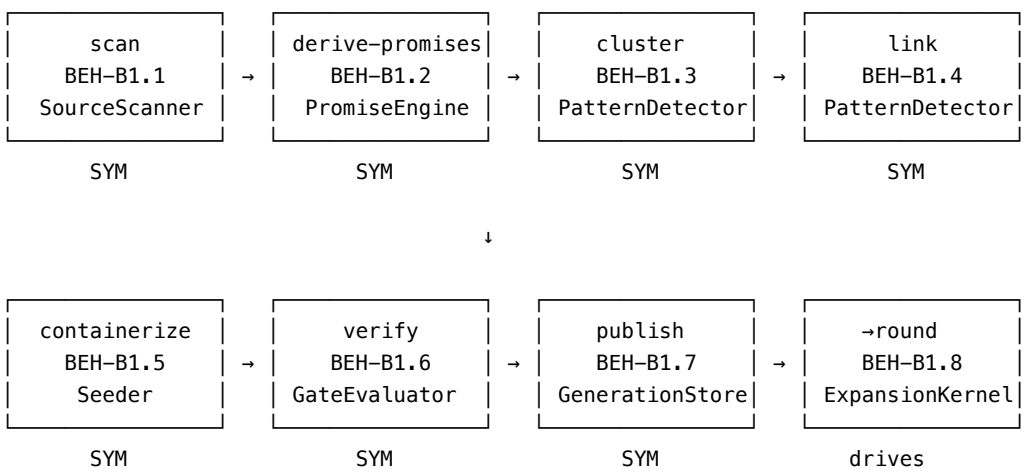
Operator vs round. The design's **one-step operator** is `select → propose → ground → admit → reconcile → bind` (one promise). The **round** is the *batched* operator: `govern` wraps `select` with a schedule, `grow` = `select`+`propose`, `gate` = batched `admit`. Full 1:1 mapping in §4.

B1 — extract_seed_graph (forks `pipeline_v3`)

Pure-AST seed pass. Forks the modern 7-stage promise-native pipeline, then hands its seed graph to `B-ROUND`. **Trigger:** Modeler, or Target System first-ingest. **Drives rounds.**

Realizes: `CAP-extract`

Flow



Promises

- **intent** *(verified)* — Produce seed nodes + grounded facts from a target's source via pure AST analysis, deterministically, no LLM. `[model]` CAP-PB intent `[context]` self-run (25 comp / 664 promises).
- **success_criteria** *(verified)* — Byte-identical across reruns (pinned clock); >=1 component per non-empty root; schema-valid; decompose reaches fixed point. `[delivery]` 4351 tests; Phase 8 byte-determinism gate.
- **requirements** *(verified)* — Deterministic AST-only; stage-composable; resumable via disk cache; emit schema-compatible YAML. `[model]` CAP-PB requirements; `tests/pipeline_v3/`.
- **failure_modes** *(verified)* — import cycles hide edge direction; name-prefix mis-clusters siblings; dynamic imports invisible to AST. `[testing]` 1030 ambiguous-module on infer (django/textual lessons).

Direct sub-behaviors (8)

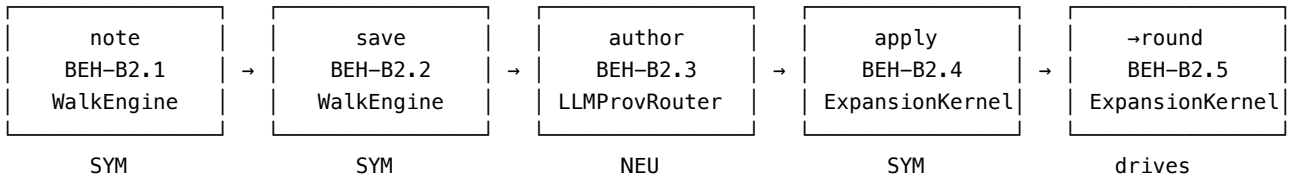
ID	Name	Intent
`BEH-B1.1`	`scan`	Pure-AST walk of the target tree; emit the observable-fact inventory (`IF-ScanReport`/Manifest).
`BEH-B1.2`	`derive-promises`	Turn facts into seed nodes carrying kind-valence open promises.
`BEH-B1.3`	`cluster`	Group modules by affinity into candidate components.
`BEH-B1.4`	`link`	Derive edges (realizes / depends-on / exposes …) between clusters.
`BEH-B1.5`	`containerize`	Wrap clusters as components allocated to layers.
`BEH-B1.6`	`verify`	Chain-closure check of the seed graph.
`BEH-B1.7`	`publish`	Emit the gen-0 `Generation` (`GEN-*`).
`BEH-B1.8`	`→round`	Feed all residual open promises into `B-ROUND`.

B2 — author_promise_content (CAP-DF ceremony)

Human-in-the-loop promise-fill: `note → save → author → apply`, reused verbatim from AMS CAP-DF and terminated by a round. **Trigger:** Modeler. **Drives rounds.**

Realizes: `CAP-model`

Flow



Promises

- **intent** *(verified)* — Support the developer inner loop: edit code, re-run extraction, browse viewer, capture a promise-note, file a change proposal — without leaving the tool suite. `[model]` BEH-28 intent.
- **requirements** *(verified)* — Notes persist under `.architecture/ comments/` with deterministic ids; Save auto-advances Job draft → approved → queued; Author walk is provider-agnostic. `[model]` BEH-28; `walks/meta/{promise_note,save,author}.py`.
- **success_criteria** *(delivered)* — Proposal applies without drift on model tip; gate passes post-apply; SI&L failure_rate not regressing. `[model]` BEH-28 success_criteria (status: open).

Direct sub-behaviors (5)

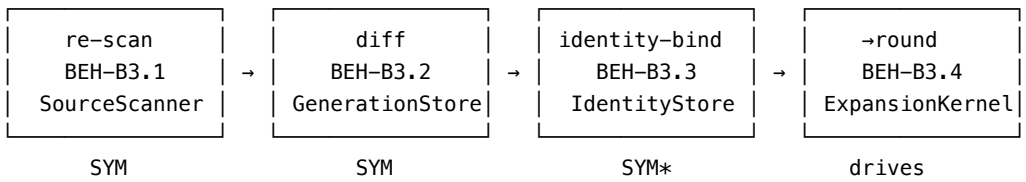
ID	Name	Intent
`BEH-B2.1`	`note`	Attach a free-text note to a promise slot `(entity, attribute)` → `NOTE-*` CommentStub.
`BEH-B2.2`	`save`	Promote the note to a `WorkOrder` (`fill_promise:<entity>:<attr>`) + tracking `Job` (auto-queued).
`BEH-B2.3`	`author`	Assemble prompt, call provider, wrap response as a `ModelPatch` Proposal (`PROP-*`).
`BEH-B2.4`	`apply`	Validate + apply the proposal to the graph (`apply_model_patch`).
`BEH-B2.5`	`→round`	Re-enter `B-ROUND` so the filled promise's downstream promises discharge.

B3 — re_extract_incremental

The incremental path: only changed subtrees re-scan, identity is preserved across generations, drift is journaled. **Trigger:** Target System change. **Drives rounds.** *No AMS pipeline counterpart — genuinely new.*

Realizes: `CAP-extract`

Flow



Promises

- **intent** *(delivered)* — Re-scan only changed paths, preserve identity across generations, journal drift, re-enter the fixpoint. `[context] architect_diff/package_diff` (diff exists); identity-bind net-new.
- **identity-bind** *(typed-gap)* — Sticky θ-crosswalk (θ_high=0.85 / θ_low=0.60) with `gap:ambiguous-identity` in the hysteresis band — net-new (lifecycle has lineage, no θ). `[plan]+[discussion]`.

Direct sub-behaviors (4)

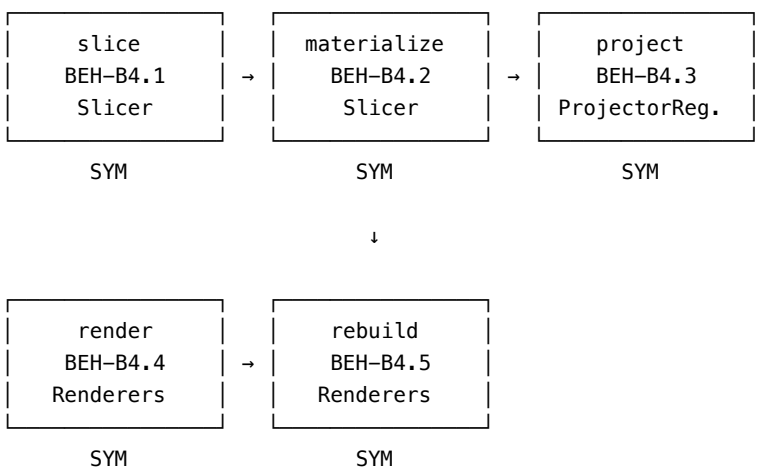
ID	Name	Intent
`BEH-B3.1`	`re-scan`	Re-scan only changed paths (reuse lifecycle `owned_paths` staleness).
`BEH-B3.2`	`diff`	Compute the semantic delta vs. the current `Generation` → `JRN-drift`.
`BEH-B3.3`	`identity-bind`	Re-bind stable IDs via θ-crosswalk; emit `gap:ambiguous-identity` in the band.
`BEH-B3.4`	`→round`	Feed re-opened promises into `B-ROUND`; emit new `GEN-∗`.

B4 — represent_artifacts (lifecycle substrate)

`slice → materialize → project → render → rebuild`, forked from the lifecycle substrate. **Trigger:** Modeler or Downstream. **Reads** the converged graph; emits owned `Artifacts`.

Realizes: `CAP-represent`

Flow



Promises

- **intent** *(verified)* — Materialize, project, and render any owned view/artifact from the graph. `[model]` CAP-SL intent `[context]` Phase 1 `ModelSlice→materialize→project→render→rebuild` substrate.
- **success_criteria** *(verified)* — Fragment digest stable across identical slice calls; zero broken cross-refs; byte-identical render. `[delivery]` lifecycle 227 tests; `test_overlay_determinism.py`.
- **requirements** *(verified)* — Respect token budget without silent truncation; no dangling refs after closure; deterministic bytes. `[model]` BEH-26 requirements.

Direct sub-behaviors (5)

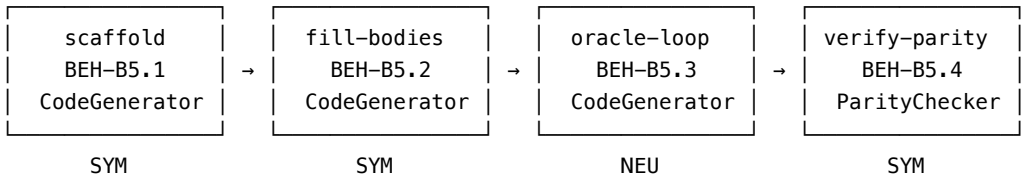
ID	Name	Intent
`BEH-B4.1`	`slice`	Resolve a `SliceSpec` into a closed fragment (select + relationship closure + curate).
`BEH-B4.2`	`materialize`	Bind the fragment to a model revision + supplementary evidence refs.
`BEH-B4.3`	`project`	Run the named projector → `DiagramSpec`; stamp freshness + provenance.
`BEH-B4.4`	`render`	Deterministically render to bytes (md / html / svg / zip / ai-context) → `ART-*`.
`BEH-B4.5`	`rebuild`	Topologically rebuild a fan-out of artifacts (per-subsystem).

B5 — generate_code_mdd (oracle-in-loop)

Model-driven code generation + behavioral parity. **Trigger:** Modeler or arch-agent. **Reads** the graph; emits code `Artifact`s and parity evidence. Precondition: an **oracle is obtainable**.

Realizes: `CAP-mdd`

Flow



Promises

- **intent** *(verified)* — Model-driven code generation + behavioral parity verification against an oracle. `[context]` Blind Regen: enriched model alone regenerates passing code for 35/35 subsystems.
- **success_criteria** *(verified)* — Blind (model-only) regen converges 100% fidelity, 1.0 avg iterations, zero source/test reads. `[context]` colorama 4/4, structlog 13/13, tqdm 10/10, click 8/8 = 35/35.
- **trade_offs** *(verified)* — Abstract model alone = 0% regen; enrichment (AST-level body_hints/constants) required for fidelity. `[context]` Regeneration (0%) vs Blind Regeneration (100%).

Direct sub-behaviors (4)

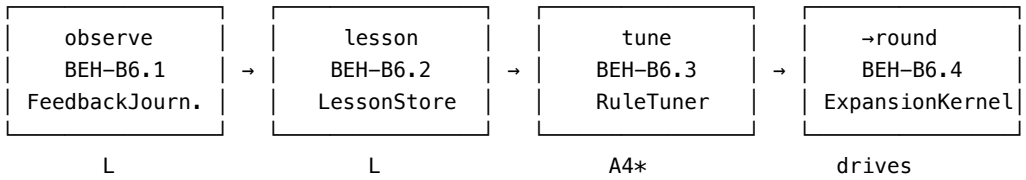
ID	Name	Intent
`BEH-B5.1`	`scaffold`	Emit module/class/function skeletons from component + interface specs → `ART-*`(code).
`BEH-B5.2`	`fill-bodies`	Fill bodies from `body_hint` / constants / test_contracts.
`BEH-B5.3`	`oracle-loop`	Run the oracle, feed failures back, converge.
`BEH-B5.4`	`verify-parity`	Diff behavior vs. oracle; score parity → `JRN-test`, `verifies` edges (A7).

B6 — learn_and_tune_rules (closes the loop)

The only behavior that feeds *rule changes* back into the engine, closing the loop AMS left open (BEH-27.5/.6). **Trigger:** Auditor or self (post-round). **Drives rounds.**

Realizes: `CAP-learn`

Flow



Promises

- **intent** *(verified)* — Observe outcomes, distill lessons, tune the rules that drive discharge. `[model]` BEH-27 intent; `pipeline/ lessons.py`, `learning.py`, `global_learning.py`.
- **requirements** *(verified)* — Persist lessons to `learning/ history.json`; dedup by content-hash; apply at stage entry not emit. `[model]` BEH-27 requirements; `pipeline/history.py`.
- **failure_modes** *(verified)* — infer chronically weak (69); allocate regressed 100→78; 1030 ambiguous-module unlearned. `[testing]` learning curve + 38 lessons.md.
- **tune (close B6)** *(delivered)* — Adjust expander/scheduler rules from lessons; re-enter B-ROUND under new rules. `[model]` BEH-27.5 `[context]` adaptive optimizer (4 heuristics).

Direct sub-behaviors (4)

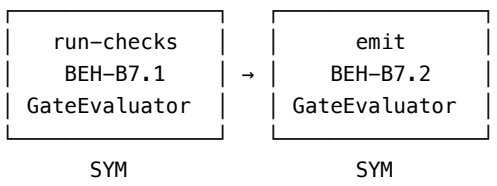
ID	Name	Intent
`BEH-B6.1`	`observe`	Append outcomes to the four journals (fail-soft) → `JRN-gates/drift/test/llm`.
`BEH-B6.2`	`lesson`	Distill deduplicated `Lesson`s from the journals (fixes AMS "lessons overwritten").
`BEH-B6.3`	`tune`	Adjust expander / scheduler rules from lessons; `affects` ExpanderRegistry, BudgetGovernor.
`BEH-B6.4`	`→round`	Re-enter `B-ROUND` under the new rules.

B7 — gate_standalone

`B-ROUND.gate` invoked on the current graph without growing it — a pure verification pass. **Trigger:** Auditor. **Is** a round stage.

Realizes: `CAP-model`

Flow



Promises

- **intent** *(verified)* — Evaluate the 3 verification classes on the current graph without mutation; emit typed gaps. `[model]` BEH-P1.7; `pipeline/validate.py`, `core/validator.py`, `authoring/gate.py`.
- **success_criteria** *(verified)* — Score >=80/100; zero false-positive orphan verdicts on well-formed models. `[delivery]` validator 90/100; `tests/test_validator.py`.

Direct sub-behaviors (2)

ID	Name	Intent
`BEH-B7.1`	`run-checks`	Evaluate existence → `gap:missing`; prohibition → `gap:violation`; parametric → fitness.
`BEH-B7.2`	`emit`	Emit `GateEvent` + typed gaps; never mutate the graph.

§4. Operator crosswalk

openAG's **one-step operator**, the AMS **walk pipeline**, and the `model.*` **L3 functions** are the *same six-stage shape viewed three ways*. This is why the existing **13 WalkSpecs + 6-stage runner** are the executable prototype of the operator — not throwaway.

operator (design §3.2)	walk stage (`walks/`)	`model.*` L3 function	round stage	seam
select	`select`	`select-promise`; build-DAG, critical-path/slack, tier-schedule, widen, batch	govern	SYM
propose	`traverse` (hop)	`propose-candidates`	grow	**NEU**
ground	`evidence`	`gather-evidence`, `check-grounding`, `score-confidence`	ground	**NEU**
admit	`classify`	`admit-schema`, `admit-constraints`	gate	SYM
reconcile	`rollup`	`dedup`, `identity-bind($\theta$)`, `roll-up-aggregate`, `canonicalize`	reconcile	SYM
bind	`emit` (cards)	`cache-decision`, `record-generation`	bind	SYM

Consequence. Because the shapes coincide, implementing the operator = generalizing the walk runner so `classify` consults the constraint set and `rollup` runs the θ-crosswalk. (Implementation deferred per design §12.)

§5. Comparison matrix vs. the 5 existing AMS pipelines

#	AMS pipeline	stages	openAG disposition	maps to	rationale
1	**Legacy 10-stage** `pipeline/`	observe → … → emit	**REPLACE (drop)**	—	Linear batch, single-shot, no fixpoint, no gap-honesty. The *only* pipeline openAG drops.
2	**v3 7-stage** `pipeline_v3/`	scan → … → publish	**FORK**	`B1 Extract`	Already promise-native; becomes the seed stage feeding `B-ROUND`.
3	**Walk 6-stage** `walks/`	select → … → emit	**REUSE**	`CAP-model.traverse` / operator §4	The walk pipeline *is* the operator's shape — kept verbatim.
4	**Lifecycle substrate**	ModelSlice → … → rebuild	**FORK**	`B4 Represent`	The single artifact substrate; projectors become registered pure projectors.
5	**CAP-DF ceremony**	note → save → author → apply	**REUSE**	`B2 Author`	Human-in-the-loop promise-fill; `apply` = `apply_model_patch`; terminated by a round.

Where the AMS operational pipelines land (report crosswalk)

The seven flat pipelines of `pipelines-v3.md` re-disposition as:

AMS report pipeline	openAG disposition	becomes
BEH-P1 10-stage	REPLACE (drop)	dissolves into `B-ROUND` round-stages
BEH-26 slice	FORK	`B4 Represent`
BEH-27 learn	FORK + close loop	`B6 Learn`
BEH-28 MDD loop	REUSE	`B2 Author`
BEH-LLM policy router	demote → cross-cutting	`LLMProviderRouter` (neural-seam provider)
BEH-SIL telemetry	demote → evidence	feeds `B-ROUND.ground` + `B6.observe`
BEH-FBK journals	demote → substrate	the 4 `JRN-*` that `B6.observe` reads

Net: openAG drops **exactly one** pipeline (legacy linear batch) and absorbs the other four + three cross-cutting services into a single `B-ROUND` fixpoint. Zero prior effort is lost (zero-loss crosswalk, entity-model §11; corpus Part 15).

Appendix: the three structural shifts (vs. AMS)

1. **Flat → fixpoint.** AMS runs 7 independent flows; openAG has 6 of them feeding or reading one shared loop (`B1/B2/B3/B6` drive; `B4/B5` read; `B7` = `gate` standalone). `B3` (incremental re-extract) is genuinely new — no AMS counterpart.
2. **Starred LLM markers promoted to first-class.** In the AMS report LLM sites are mostly `SI&L` with two *proposed* `LLM*` (synthesize `BEH-P1.9`, `propose_patches BEH-27.6`). openAG makes exactly those the **neural seam**: `grow` and `ground` are the *only* two stages where a model may invent/score structure — and it never writes directly.
3. **"open" → typed-gap (gap-honesty).** The AMS report's 18% unverified (failure_modes / success_criteria open on BEH-P1/26/28; BEH-LLM.6/.7 cost-cap/fallback open) stop being silent "open" status. Under invariant #5, anything ungrounded within budget is emitted as an explicit `typed-gap` — the corpus already parks BEH-LLM.6/.7 in its §12 reservoir for exactly this reason.

Net-new invention surface (typed-gaps, corpus §13)

Nine primary promises have no prior AMS source — the genuine openAG invention: `ExpanderRegistry` total `(kind,axis)` registry · `Grounder` candidate-admission-against-evidence · `IdentityStore` θ-crosswalk · `SeedStrategy` · the `axis+valence` promise fields · the `EvidenceRef` typed taxonomy (static/observational/executable) · `CON-N2` NN-compilable axis-typed round-trip. Everything else is forked or reused from AMS.